EXPERIMENT 4

Title: Finite-State Morphological Parsing for Plural Noun Formation

Aim:

To develop a Python program that performs simple morphological processing by converting singular English nouns into their plural forms using rule-based finite-state processing. Morphological parsing examines the structure of words and applies language rules to produce the required word forms. In this experiment, rules are used to handle regular plural formation, including nouns ending in consonant + y and nouns ending in s, x, ch, or sh.

Algorithm:

1. Start the program.
2. Define a function named make_plural() that accepts a singular noun.
3. Check whether the word ends with y and the preceding character is a consonant.
4. If the condition is satisfied, remove y and add ies.
5. Otherwise, check whether the word ends with s, x, ch, or sh.
6. If it matches one of these endings, add es to the word.
7. If neither condition is satisfied, add s to form the plural.
8. Create a list containing sample singular nouns.
9. Apply the function to each word in the list.
10. Display each singular noun and its corresponding plural form.
11. Stop the program.

Program:

Exp-04-Finite-State Machine for Plural Formation.py - C:\Users\jerin\OneDrive\Desktop\NLP...

File Edit Format Run Options Window Help

```
def plural(word):
    if word.endswith("y") and word[-2] not in "aeiou":
        return word[:-1] + "ies"

    elif word.endswith(("s", "x", "ch", "sh")):
        return word + "es"

    else:
        return word + "s"


words = ["cat", "bus", "box", "watch", "baby"]

for word in words:
    print(word, "->", plural(word))
```

Output:

```
>>>
= RESTART: C:\Users\jerin\OneDrive\Desktop\NLP\
achine for Plural Formation.py
cat -> cats
bus -> buses
box -> boxes
watch -> watches
baby -> babies
>>>
```

Result:

Thus, a rule-based morphological processing program was implemented successfully to generate plural nouns from singular nouns. The program applies different rules according to the endings of the input words. This experiment demonstrates how morphological rules can be represented computationally in NLP. The rules cover common patterns and do not handle every irregular noun, such as child -> children.